# risk_free_curve_projection

November 4, 2026

## 1 Projection of the risk free curve and recalibration

### 1.1 Table of contents


1. Importing files
2. Preparation and treatment of the curves
   - The curves class
   - Save the input term structure
   - Calculate 1 year forward rates
   - Forward yield curve
3. Calibrate Smith-Wilson on each yield curve
   - Calibration of the interpolation/extrapolation algorithm
   - Does the input curve's length explain it?
   - Spread treatment
4. Example 1: the algorithm retains the liquid points
5. Example 2: yields for arbitrary maturities
   - Yield calculation
   - Capitalisation factor calculation
   - Discount factor calculation


The purpose of this workbook is to showcase the calibration methodology inside OSEM. The key to the calibration is the Smith Wilson algorithm commonly used in insurance. This algorithm allows a continuous approximation of arbitrary maturities based on a subset of available maturities. This example is split onto 3 parts.

- The first part shows how the necessary parameters and curves are imported from input files.
- Second part shows how the curves are generated and the Smith-Wilson calibrations derived.
- The last part shows how the saved calibration can be used to derive the discount rates for any maturity.

[1]:
```
import datetime
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
from pathlib import Path
import sys
```

```
base_folder = next(path for path in [Path.cwd(), *Path.cwd().parents] if (path /
  ↪ "ALM.ini").is_file())
sys.path.insert(0, str(base_folder / "src"))
import os
os.chdir(base_folder)
```

[2]:
```
from osem.ImportData import import_SWEiopa
from osem.CurvesClass import Curves
```

## 1.2 1. Importing files

The parameters and the current risk free curve are provided as input:

- Parameters.csv; Parameters related to the run
- EIOPA_param_file.csv; Assumed yield curve at modelling date and relevant maturities
- EIOPA_curves_file.csv; Parameters related to the EIOPA time 0 calibration

The exact location of the EIOPA files is encoded in the Parameters input file.

[3]:
```
# The notebooks' fixed copy of Parameters.csv, which names their copies of the␣
  ↪EIOPA files
paramfile = pd.read_csv("notebooks/data/Input/Parameters.csv",␣
  ↪index_col="Parameter")
```

The location of the two EIOPA files are:

[4]:
```
selected_param_file = paramfile["Value"].loc["EIOPA_param_file"]
selected_curves_file = paramfile["Value"].loc["EIOPA_curves_file"]
```

The risk free curve belongs to the following country:

[5]:
```
country = paramfile["Value"].loc["country"]
```

Import all necessary parameters:

[6]:
```
[maturities_country, curve_country, extra_param, Qb] =␣
  ↪import_SWEiopa(selected_param_file, selected_curves_file, country)
```

## 1.3 2. Preparation and treatment of the curves

The OSEM does multiple steps in order to make it easier for OSEM to access discount curves at any point in time and for arbitrary maturities. All steps are performed and saved within the Curves class.

### 1.3.1 The curves class

The curve class object contains all the data necessary to run the model.

[7]:
```
# ultimate forward rate
ufr = extra_param["UFR"]/100
```

```
# Numeric precision of the optimisation
precision = float(paramfile["Value"].loc["Precision"])

# Targeted distance between the extrapolated curve and the ultimate forward␣
 ↪rate at the convergence point
tau = float(paramfile["Value"].loc["Tau"])# 1 basis point

# A real date, as main.py passes. Note that Curves stores country but never uses
# initial_date, so this value does not affect any result.
modelling_date = datetime.datetime.strptime(
    paramfile["Value"].loc["Modelling_Date"], "%d/%m/%Y").date()

# Number of projection years
n_years = int(paramfile.loc["n_proj_years"].values[0])
```

[8]:
```
curves = Curves(ufr, precision, tau, modelling_date, country)
```

After the Curves class is initiated, there are a series of calibrations that needs to be performed.

- Saving of the input term structure
- Calculation of the forward rates
- Projection of the forward spot rates
- Calculation of the Smith-Wilson calibrations for each period

### 1.3.2 Save the input term structure

SetObservedTermStructure sets the liquid maturities and the corresponding yields to the m_obs and r_obs property of the Curves class.

[9]:
```
curves.set_observed_term_structure(maturity_vec=curve_country.index.
 ↪to_numpy(dtype=float), yield_vec=curve_country.values)
```

The term structure that every later step is built from:

---

[10]:
```
input_curve = pd.DataFrame({"Maturity": curves.m_obs_ini["Maturity"],
                            "Yield": curves.r_obs_ini["Yield"]})

display(input_curve.head(10))
```

| | Maturity | Yield |
|---|---|---|
| 0 | 1.0 | 0.03271 |
| 1 | 2.0 | 0.03439 |
| 2 | 3.0 | 0.03470 |
| 3 | 4.0 | 0.03482 |
| 4 | 5.0 | 0.03488 |
| 5 | 6.0 | 0.03497 |
| 6 | 7.0 | 0.03512 |
| 7 | 8.0 | 0.03532 |

```
8       9.0  0.03553
9      10.0  0.03577
```

---

### 1.3.3 Calculate 1 year forward rates

Using the time 0 spot curve provided as input, the yearly forward rates can be calculated from the annually reported yield curve. `calc_fwd_rates` stores the **capitalisation factor** $1 + fw_i$ rather than the rate itself, so the column `fwd_rates["Forward"]` holds values around 1.03, not 0.03.

For the first year:

$$1 + fw_{EIOPA,1} = 1 + y_1$$

otherwise:

$$1 + fw_{EIOPA,i} = \frac{(1 + y_i)^i}{(1 + y_{i-1})^{i-1}}$$

Note that this implementation assumes that all forward rates are exactly 1 year apart as provided by EIOPA.

Where:

- $i$ is the year (Ex. 1,2,3, ...)
- $y_i$ is the yield for the year $i$
- $fw_{EIOPA,i}$ is the 1-year forward rate between years $i - 1$ and $i$

[11]:
```
curves.calc_fwd_rates()
```

Note the third column: the stored values sit around 1.03, because `fwd_rates` holds the capitalisation factor $1 + fw$ rather than the rate itself.

---

[12]:
```
forward_table = pd.DataFrame({"Maturity": curves.m_obs_ini["Maturity"],
                              "Input yield": curves.r_obs_ini["Yield"],
                              "1 + forward": curves.fwd_rates["Forward"]})
forward_table["Forward rate"] = forward_table["1 + forward"] - 1

display(forward_table.head(10))
```

```
   Maturity  Input yield  1 + forward  Forward rate
0       1.0      0.03271     1.032710      0.032710
1       2.0      0.03439     1.036073      0.036073
2       3.0      0.03470     1.035320      0.035320
3       4.0      0.03482     1.035180      0.035180
4       5.0      0.03488     1.035120      0.035120
5       6.0      0.03497     1.035420      0.035420
6       7.0      0.03512     1.036020      0.036020
```

```
7        8.0       0.03532      1.036721       0.036721
8        9.0       0.03553      1.037212       0.037212
9       10.0       0.03577      1.037933       0.037933
```

---

### 1.3.4 Forward yield curve

The forward yield curve can be calculated by using the 1-year forward rates from the suitable moment onwards. The spot yield curve is calculated for each projection period. For the projection period $i$, the yield curve can be calculated as:

$$1 + y_m^j = \left( \prod_{k=j+1}^{j+m} (1 + fw_{EIOPA}(k)) \right)^{1/m}$$

The spot rate is the geometric mean of the capitalisation factors covering the period, less one. In particular the shortest point of each projected curve is just the forward rate for that year, $y_1^j = fw_{EIOPA}(j+1)$.

Where: - $j$ is the modelling period at which the model is performing the calculations - $m$ is the maturity measured from the start of modelling period $j$ - $y_m^j$ is the yield $m$ years after the modelling period $j$ - $fw_{EIOPA}(k)$ is the 1-year forward rate covering the year from $k-1$ to $k$

[13]:
```
# The +1 matches main.py: the projected curve has to reach one year past the␣
 ↪end of
# the modelling window so the last projection period can still be priced.
curves.project_forward_rate(n_years+1)
```

Two properties the projection has to satisfy. Year 0 is the input curve itself, and the shortest point of each projected curve is the 1-year forward rate covering that year:

---

[14]:
```
projection_check = pd.DataFrame({"Input yield": curves.r_obs_ini["Yield"],
                                 "Projected year 0": curves.
 ↪r_obs["Yield_year_0"]})
projection_check["Difference"] = (projection_check["Projected year 0"]
                                  - projection_check["Input yield"])

one_year = pd.DataFrame(
    {"1-year spot of year j": [curves.r_obs["Yield_year_" + str(j)].dropna().
 ↪values[0]
                               for j in range(1, 6)],
     "Forward rate j+1": [curves.fwd_rates["Forward"].values[j] - 1 for j in␣
 ↪range(1, 6)]},
    index=["year " + str(j) for j in range(1, 6)],
)
one_year["Difference"] = one_year["1-year spot of year j"] - one_year["Forward␣
 ↪rate j+1"]
```

```
display(projection_check.head(8))
display(one_year)
```

```
   Input yield  Projected year 0    Difference
0      0.03271           0.03271  1.387779e-17
1      0.03439           0.03439 -7.632783e-17
2      0.03470           0.03470 -4.857226e-17
3      0.03482           0.03482  7.632783e-17
4      0.03488           0.03488  2.081668e-17
5      0.03497           0.03497 -5.551115e-17
6      0.03512           0.03512  4.163336e-17
7      0.03532           0.03532  2.081668e-17
```

| | 1-year spot of year j | Forward rate j+1 | Difference |
|---|---|---|---|
| year 1 | 0.036073 | 0.036073 | 0.0 |
| year 2 | 0.035320 | 0.035320 | 0.0 |
| year 3 | 0.035180 | 0.035180 | 0.0 |
| year 4 | 0.035120 | 0.035120 | 0.0 |
| year 5 | 0.035420 | 0.035420 | 0.0 |

---

## 1.4 3. Calibrate Smith-Wilson on each yield curve

Each forward yield curve is used to calibrate the SW algorithm. This calibration can then be used to generate yield curves of arbitrary maturities. The calibration is split on two parts. First one calibrates the curve at time 0.

### 1.4.1 Calibration of the interpolation/extrapolation algorithm

Both at time 0 and in following projection years, the same calibration algorithm is performed. The algorithm looks to minimize the difference between the extrapolated forward intensity at the convergence point and the ultimate forward intensity, as recommended by EIOPA:

$$T = \max(U + 40, 60)$$

$$g(\alpha) = |f(T) - \omega| = \frac{\alpha}{|1 - \kappa e^{\alpha T}|}$$

The calibration consists of using a bisection algorithm to minimize the difference between the ultimate forward rate $\omega$.

The calibration is:

$$\min \alpha$$

subject to constraints:

$$\alpha \geq \alpha_{min}$$

and

$$g(\alpha) \leq \tau$$

And $\kappa$ is defined as:

$$\kappa = \frac{1 + \alpha u^T Q b}{sinh(\alpha u^T) Q b}$$

$$Q = dIC$$

$$d = e^{-\omega u}$$

Where: - $\alpha$ is the free parameter to be calibrated - $\alpha_{min}$ is the lower bound of the allowable values - $U$ is the last liquid point, the longest maturity in the input curve - $T$ is the convergence point at which the extrapolated curve must have reached the ultimate forward rate - $g(\alpha)$ is the gap at the convergence point between the extrapolated forward intensity $f(T)$ and the ultimate forward intensity $\omega$ - $\omega = \ln(1 + ufr)$ is the ultimate forward intensity - $\tau$ is the largest gap that is still accepted at the convergence point - $u$ is the vector of liquid maturities used in the calibration - $C$ is a matrix of cash flows, the identity matrix here since the inputs are treated as zero-coupon bonds with a notional of 1 - $d$ is the vector of ultimate-forward discount factors at those maturities - $b$ is the Smith-Wilson calibration vector returned by `sw_calibrate`

In the implementation, `g_alfa` returns $g(\alpha) - \tau$ and `bisection_alpha` finds its root, that is the $\alpha$ at which the constraint binds. Because the gap shrinks as $\alpha$ grows, that root is the smallest admissible $\alpha$.

[15]:
```
curves.calibrate_projected(n_years+1, 0.05, 0.5, 1000)
```

The calibration produces one convergence speed $\alpha$ per projection year.

Every year here returns `0.05`, the lower end of the bracket passed to `calibrate_projected`. That is the correct answer rather than a failure: with this input $g(\alpha)$ stays below $\tau$ across the whole bracket, so the constraint $g(\alpha) \leq \tau$ is satisfied everywhere and the EIOPA rule *minimise* $\alpha$ selects the smallest admissible value. `bisection_alpha` detects that case explicitly; it used to walk to the far end of the bracket and return `0.5`, the largest value, instead. The next subsection shows what drives which case you land in.

---

[16]:
```
alpha_table = pd.DataFrame(
    {"Alpha": [curves.alpha["Alpha_year_" + str(j)][0] for j in range(n_years +␣
↪1)]},
    index=["year " + str(j) for j in range(n_years + 1)],
)
```

```
display(alpha_table.head(10))
print("distinct calibrated alphas:", sorted(alpha_table["Alpha"].unique()))
```

```
        Alpha
year 0   0.05
year 1   0.05
year 2   0.05
year 3   0.05
year 4   0.05
year 5   0.05
year 6   0.05
year 7   0.05
year 8   0.05
year 9   0.05

distinct calibrated alphas: [np.float64(0.05)]
```

---

### 1.4.2 Does the input curve's length explain it?

The convergence point is $T = \max(U + 40, 60)$, so a long last liquid point $U$ pushes $T$ far out and the extrapolation has already reached the ultimate forward rate whatever $\alpha$ is. The table below rebuilds the calibration on the input curve truncated at a range of last liquid points, using throwaway `Curves` objects so nothing above or below is affected.

`Bracketed` is the condition the bisection needs: $g$ must change sign across $[\alpha_{min}, \alpha_{max}]$ for a root to exist.

---

[17]:
```
alpha_by_last_point = []
maturities_all = curve_country.index.to_numpy(dtype=float)

for last_point in (150, 100, 70, 60, 50, 40, 30, 20):
    keep = maturities_all <= last_point
    m_cut = maturities_all[keep]
    r_cut = curve_country.values[keep]

    probe = Curves(ufr, precision, tau, modelling_date, country)
    probe.set_observed_term_structure(maturity_vec=m_cut, yield_vec=r_cut)

    alpha_by_last_point.append({
        "Last liquid point": last_point,
        "Convergence point T": max(last_point + 40, 60),
        "g(0.05)": probe.g_alfa(m_cut, r_cut, probe.ufr, 0.05, tau),
        "g(0.50)": probe.g_alfa(m_cut, r_cut, probe.ufr, 0.50, tau),
        "Calibrated alpha": probe.bisection_alpha(0.05, 0.5, m_cut, r_cut,
                                                  probe.ufr, tau, precision,␣
 ↪1000),
```

```
    })

alpha_diagnostic = pd.DataFrame(alpha_by_last_point).set_index("Last liquid␣
 ↪point")
alpha_diagnostic["Bracketed"] = (np.sign(alpha_diagnostic["g(0.05)"])
                                 != np.sign(alpha_diagnostic["g(0.50)"]))

display(alpha_diagnostic)
```

| Last liquid point | Convergence point T | g(0.05) | g(0.50) | Calibrated alpha | Bracketed |
|---|---|---|---|---|---|
| 150 | 190 | -0.000055 | -0.0001 | 0.05 | False |
| 100 | 140 | -0.000061 | -0.0001 | 0.05 | False |
| 70 | 110 | -0.000026 | -0.0001 | 0.05 | False |
| 60 | 100 | -0.000004 | -0.0001 | 0.05 | False |
| 50 | 90 | -0.000034 | -0.0001 | 0.05 | False |
| 40 | 80 | -0.000020 | -0.0001 | 0.05 | False |
| 30 | 70 | -0.000054 | -0.0001 | 0.05 | False |
| 20 | 60 | -0.000015 | -0.0001 | 0.05 | False |

---

Truncating the curve does **not** change the situation for this input: at every last liquid point in the table, down to the 20 years EIOPA uses for the euro, $g$ is negative at both ends of the bracket, so there is no root and the calibration returns the lower bound. EIOPA reaches the same answer: its published $\alpha$ for this curve, the `alpha` row of `Param_no_VA.csv`, is also 0.05. On another curve $g$ can change sign inside the bracket, and $\alpha$ is then determined by the data rather than by the edge of the bracket.

Note that truncating the input for real is not an option here. The projection consumes one maturity per year, so a curve of 50 points cannot support 51 projection years: `project_forward_rate` runs out of maturities and `calibrate_projected` raises `ValueError: max() iterable argument is empty`. The input curve has to be longer than the projection horizon, which is why it runs to 150 years in the first place.

### 1.4.3 Spread treatment

`retrieve_rates` takes a `spread` argument and adds it to the extrapolated yield as a parallel shift, after the Smith-Wilson extrapolation rather than inside it:

$$y_{spread}(v) = y(v) + s$$

Every quantity derived from the yield therefore carries the shift too, so the capitalisation and discount factors become $(1+y(v)+s)^v$ and $(1+y(v)+s)^{-v}$. This is the mechanism the asset classes use to price with a credit spread: bonds pass their calibrated z-spread, equities the sum of their country, sector and stress spreads. Every example below uses `spread = 0` and therefore prices off the risk free curve alone.

## 1.5 4. Example 1: the algorithm retains the liquid points

One of the properties of the Smith-Wilson algorithm is that it retains the curve values at the liquid points provided in the calibration. This property is tested in the first example. A projection year is selected and the algorithm is asked to provide the values using the calibration vector. The original yield curve that was used to calibrate is then compared to the output.

[18]:
```
spread = 0.00
```

[19]:
```
p_year = 6
```

[20]:
```
interpolated_curve = curves.retrieve_rates(p_year, curves.
↪m_obs["Maturities_year_"+str(p_year)].values[:-p_year],␣
↪"Yield",spread=spread)
```

Quantifying what the chart below shows:

---

[21]:
```
example_1_check = pd.DataFrame({
    "Maturity": curves.m_obs["Maturities_year_" + str(p_year)].values[:-p_year],
    "Provided": curves.r_obs["Yield_year_" + str(p_year)].values[:-p_year],
    "Recalculated": np.asarray(interpolated_curve).ravel(),
})
example_1_check["Difference"] = (example_1_check["Recalculated"]
                                 - example_1_check["Provided"])

display(example_1_check.head(10))
print(f"max |difference| over all {len(example_1_check)} liquid points: "
      f"{example_1_check['Difference'].abs().max():.3e}")
```

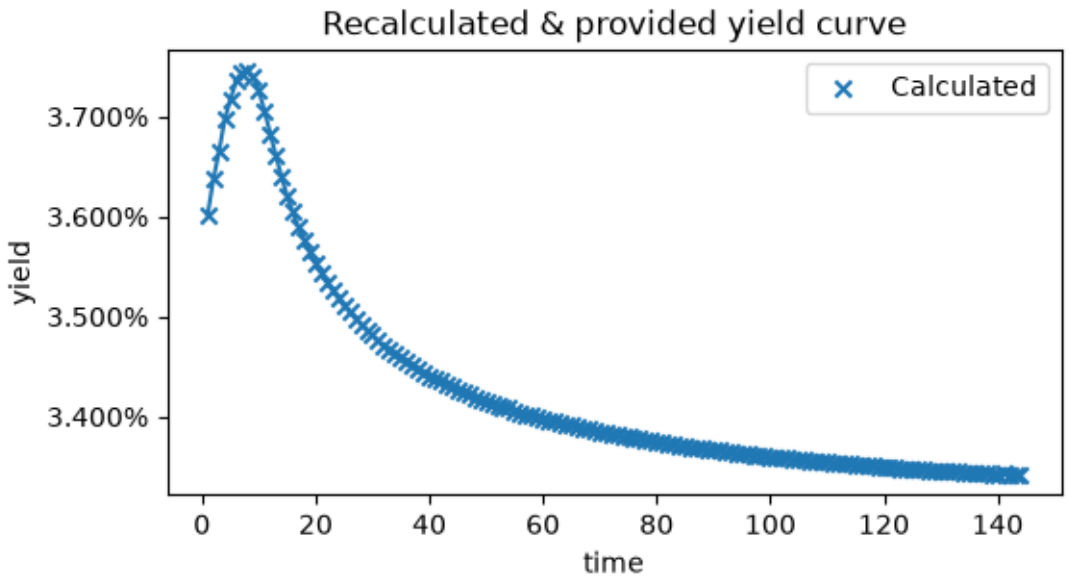
| | Maturity | Provided | Recalculated | Difference |
|---|---|---|---|---|
| 0 | 1.0 | 0.036020 | 0.036020 | 1.956213e-13 |
| 1 | 2.0 | 0.036371 | 0.036371 | 2.733369e-13 |
| 2 | 3.0 | 0.036651 | 0.036651 | 2.069456e-13 |
| 3 | 4.0 | 0.036971 | 0.036971 | 1.847411e-13 |
| 4 | 5.0 | 0.037171 | 0.037171 | 1.845191e-13 |

```
5       6.0  0.037351      0.037351  1.849632e-13
6       7.0  0.037441      0.037441  1.687539e-13
7       8.0  0.037456      0.037456  1.665335e-13
8       9.0  0.037404      0.037404  1.447731e-13
9      10.0  0.037259      0.037259  1.474376e-13

max |difference| over all 144 liquid points: 2.733e-13
```

---

[22]:
```
fig, ax1 = plt.subplots(1, 1)
ax1.scatter(curves.m_obs["Maturities_year_"+str(p_year)].values[:-p_year],␣
 ↪interpolated_curve*100, label="Calculated", marker="x")
ax1.plot(curves.m_obs["Maturities_year_"+str(p_year)].values[:-p_year], curves.
 ↪r_obs["Yield_year_"+str(p_year)].values[:-p_year]*100)
ax1.set_ylabel("yield")
ax1.set_title('Recalculated & provided yield curve')
ax1.set_xlabel("time")
ax1.legend()
ax1.yaxis.set_major_formatter(mtick.PercentFormatter())
fig.set_figwidth(6)
fig.set_figheight(3)
plt.show()
```




---

## 1.6 5. Example 2: yields for arbitrary maturities

The calibration will be used to provide OSEM with yields for arbitrary maturities. This example verifies if the calibration provides yields for maturities not provided as input into the calibration. Assuming the algorithm is processing the year 3. A hypothetical asset has a cash flow at year fractions:

$$\{0.7, 1.2, 1.3543\}$$

To make it possible to calculate the present value, the corresponding discount rates are calculated:

With the calibrated vector $b$, the required yields can easily be obtained by applying the formula:

$$p(v) = e^{-\omega v}(1 + H(v, u)Qb)$$

and converting the resulting prices to annualised yields:

$$y(v) = p(v)^{-1/v} - 1$$

Where: - $v$ is the vector of desired maturities - $ $ is the ultimate forward intensity - $u$ is the vector of maturities that were used in the calibration - $H(v, u)$ is the Heart of the Wilson function - $Q$ and $b$ are as defined in the calibration section above

[23]:
```
desired_mat = np.array([0.7, 1.2, 1.3543])
```

### 1.6.1 Yield calculation

The yield in each modelling period can be interpolated/extrapolated using the Smith-Wilson calibration vector calculated above. This allows OSEM to apply continuous yields.

[24]:
```
spread = 0.0
```

---

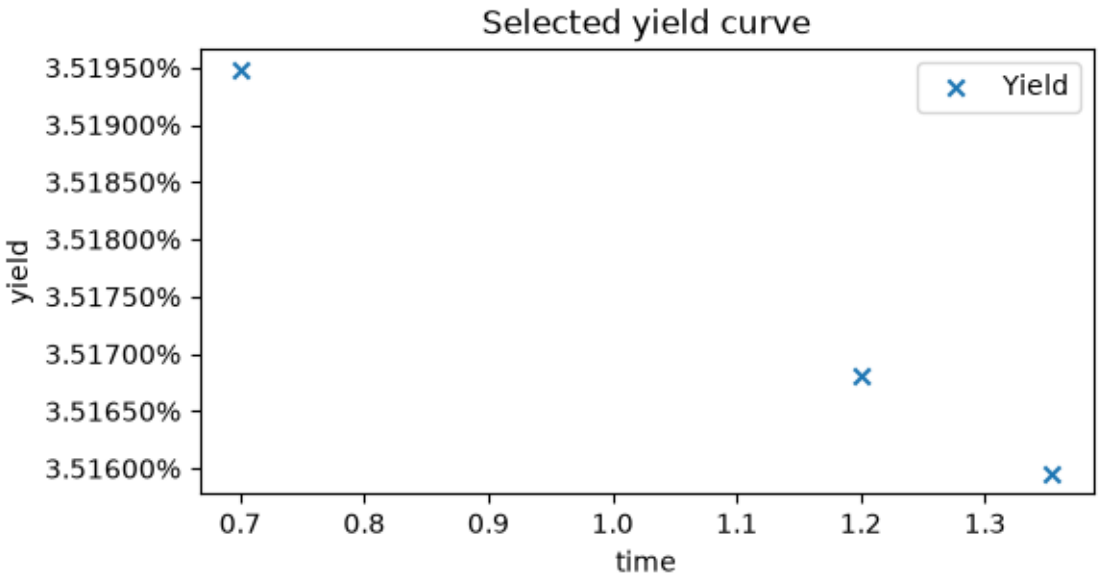
[25]:
```
display(curves.retrieve_rates(3, desired_mat, "Yield",spread))
```

```
      Yield
0  0.035195
1  0.035168
2  0.035160
```

---

[26]:
```
fig, ax1 = plt.subplots(1, 1)
ax1.scatter(desired_mat, curves.retrieve_rates(3, desired_mat, "Yield",␣
 ↪spread)*100, label="Yield", marker="x")
ax1.set_ylabel("yield")
ax1.set_title('Selected yield curve')
ax1.set_xlabel("time")
ax1.legend()
```

```
ax1.yaxis.set_major_formatter(mtick.PercentFormatter())
fig.set_figwidth(6)
fig.set_figheight(3)
plt.show()
```




---

### 1.6.2 Capitalisation factor calculation

The capitalisation factor can be calculated as:

$$c_i(t) = (1 + y_i(t))^t$$

---

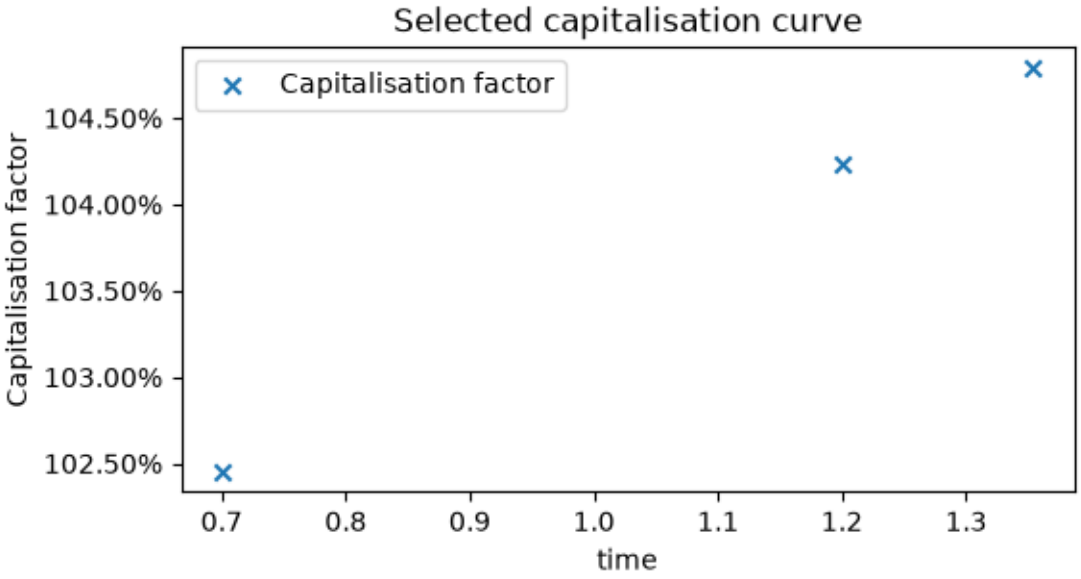
[27]:
```
display(curves.retrieve_rates(3, desired_mat, "Capitalisation", spread))
```

| | Capitalisation |
|---|---|
| 0 | 1.024508 |
| 1 | 1.042349 |
| 2 | 1.047911 |

---

[28]:
```
fig, ax1 = plt.subplots(1, 1)
ax1.scatter(desired_mat, curves.retrieve_rates(3, desired_mat,␣
 ↪"Capitalisation", spread)*100, label="Capitalisation factor", marker="x")
ax1.set_ylabel("Capitalisation factor")
ax1.set_title('Selected capitalisation curve')
```

```
ax1.set_xlabel("time")
ax1.legend()
ax1.yaxis.set_major_formatter(mtick.PercentFormatter())
fig.set_figwidth(6)
fig.set_figheight(3)
plt.show()
```




---

### 1.6.3 Discount factor calculation

The discount factors can be calculated from the yield:

$$d_i(t) = (1 + y_i(t))^{-t}$$

Where: - $t$ is time increment from the modelling period (Ex. 1.5 is 1 year and 6 months after the modelling period - $y_i(t)$ is the annualized yield for maturity t after the modelling period $i$ - $c_i(t)$ is the capitalisation factor for 1 unit, until $t$ years after the modelling period $i$

- $d_i(t)$ is the discount factor for a cash flow of value 1, occurring $t$ years after the modelling period $i$, discounted to the modelling period $i$

---

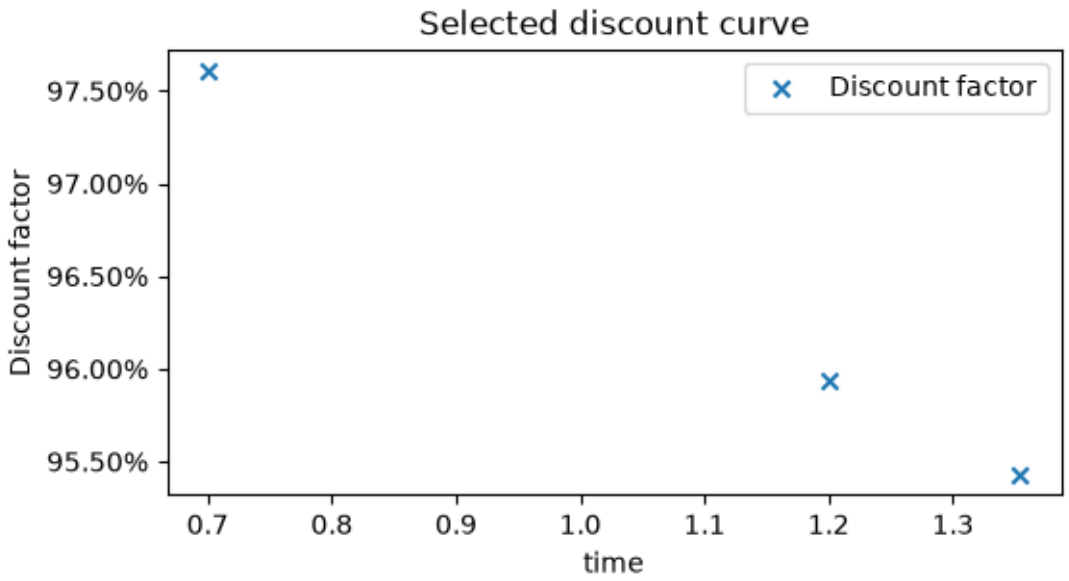
[29]:
```
display(curves.retrieve_rates(3, desired_mat, "Discount", spread))
```

```
   Discount
0  0.976078
```

```
1   0.959372
2   0.954280
```

---

[30]:
```
fig, ax1 = plt.subplots(1, 1)
ax1.scatter(desired_mat, curves.retrieve_rates(3, desired_mat, "Discount",␣
 ↪spread)*100, label="Discount factor", marker="x")
ax1.set_ylabel("Discount factor")
ax1.set_title('Selected discount curve')
ax1.set_xlabel("time")
ax1.legend()
ax1.yaxis.set_major_formatter(mtick.PercentFormatter())
fig.set_figwidth(6)
fig.set_figheight(3)
plt.show()
```




---